```
package com.navox.agent;

import
android.accessibilityservice.AccessibilityS
ervice;
import android.content.Context;
import android.content.Intent;
import android.os.Handler;
import android.os.Looper;
import android.provider.Settings;
import
android.view.accessibility.AccessibilityEve
nt;
import
android.view.accessibility.AccessibilityNod
eInfo;
import java.util.ArrayList;
import java.util.List;

public class NavoxAccess extends
AccessibilityService {
    static volatile NavoxAccess inst =
null;
    final Handler h = new
Handler(Looper.getMainLooper());

    @Override
    protected void onServiceConnected() {
        inst = this;
    }
```

```
    @Override
    public void
onAccessibilityEvent(AccessibilityEvent e)
{ }

    @Override
    public void onInterrupt() { }

    @Override
    public boolean onUnbind(Intent i) {
        inst = null;
        return super.onUnbind(i);
    }

    static boolean global(int a) {
        NavoxAccess s = inst;
        return s != null &&
s.performGlobalAction(a);
    }

    static boolean open(Context c, String
action) {
        try {
            Intent i = new Intent(action);

i.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK);
            c.startActivity(i);
            return true;
        } catch (Exception e) {
            return false;
        }
    }
```

```
    static String toggle(Context c, String
what, int want) {
        String action;
        String label;
        if (what.equals("wifi")) {
            action =
"android.settings.panel.action.WIFI";
            label = "Wi-Fi";
        } else if
(what.equals("bluetooth")) {
            action =
Settings.ACTION_BLUETOOTH_SETTINGS;
            label = "Bluetooth";
        } else {
            action =
Settings.ACTION_AIRPLANE_MODE_SETTINGS;
            label = "Airplane mode";
        }
        boolean opened = open(c, action);
        if (!opened && what.equals("wifi"))
opened = open(c,
Settings.ACTION_WIFI_SETTINGS);
        if (!opened) return "Could not open
" + label + " settings.";
        if (want == 2) return "Opened " +
label + ". Say on or off to switch it.";
        NavoxAccess s = inst;
        if (s == null) return "Opened " +
label + ". Turn on NAVOX in Accessibility
settings to switch it automatically.";
        s.clickSwitch(want, 0);
        return label + (want == 1 ? " on."
: want == 0 ? " off." : " toggled.");
    }
```

```
    void clickSwitch(final int want, final
int attempt) {
        h.postDelayed(new Runnable() {
            public void run() {
                boolean done = false;
                try {
                    AccessibilityNodeInfo
root = getRootInActiveWindow();
                    if (root != null) {

AccessibilityNodeInfo sw =
findSwitch(root);
                        if (sw != null) {
                            boolean isOn =
sw.isChecked();
                            if (want == -1
|| (want == 1) != isOn) clickNode(sw);
                            done = true;
                        }
                    }
                } catch (Exception e) { }
                if (done) {
                    h.postDelayed(new
Runnable() {
                        public void run() {

performGlobalAction(GLOBAL_ACTION_BACK);
                        }
                    }, 900);
                } else if (attempt < 8) {
                    clickSwitch(want,
attempt + 1);
                }
```

```
            }
        }, attempt == 0 ? 900 : 450);
    }

    static AccessibilityNodeInfo
findSwitch(AccessibilityNodeInfo root) {
        List<AccessibilityNodeInfo> q = new
ArrayList<AccessibilityNodeInfo>();
        q.add(root);
        int i = 0;
        while (i < q.size() && i < 500) {
            AccessibilityNodeInfo n =
q.get(i++);
            if (n == null) continue;
            if (n.isCheckable() &&
n.isVisibleToUser()) return n;
            for (int k = 0; k <
n.getChildCount(); k++) {
                AccessibilityNodeInfo ch =
n.getChild(k);
                if (ch != null) q.add(ch);
            }
        }
        return null;
    }

    static void
clickNode(AccessibilityNodeInfo n) {
        AccessibilityNodeInfo p = n;
        int d = 0;
        while (p != null && d < 6) {
            if (p.isClickable()) {

p.performAction(AccessibilityNodeInfo.ACTIO
```

```
N_CLICK);
                return;
            }
            p = p.getParent();
            d++;
        }
    }
}
```